Recommendation Pipeline: Final Report

Scope. A reproducible batch pipeline ingests, validates and transforms user, movie and interaction records, trains two ranking models, and compares their recommendations. The included data is synthetic; results demonstrate the workflow and are not evidence of production performance.

Data and design

The seeded generator creates 120 users, 100 items in five categories, and 4,200 unique user-item interactions. Each user has two preferred categories that influence sampled items and ratings. Events span 300 days starting January 1, 2024. Three CSVs are the input contract; the same workflow accepts external CSVs through configuration.

Validation checks required values, numeric age and rating ranges, timestamp parsing, duplicate entity and event keys, and foreign keys against accepted users and items. Rejected records and reason codes are quarantined in separate CSVs. The sample run accepted all 120 users, 100 items and 4,200 interactions. The clean dataset does not demonstrate the rejection path, although checks and error datasets are implemented.

A global UTC cutoff of August 1, 2024 creates 3,021 training and 1,179 test events. Only pre-cutoff interactions feed feature computation and models. User activity and mean rating, preferred category and recency; item popularity and mean rating; and pair-level count, rating and recency are saved as reusable feature tables. The feature catalog documents keys and missing-value policies.

Models and workflow

The baseline orders items by training event count. The personalized model applies rank-12 truncated singular value decomposition to a binary implicit user-item matrix and ranks reconstructed scores. Both exclude training-seen items; unknown users use popularity. A dependency-ordered Python entry point runs all stages and stops on error. Per-run JSON records configuration, timings, input hashes and metrics; successful runs snapshot model and evaluation artifacts.

Evaluation and limitations

Test relevance consists of distinct post-cutoff items not seen by that user in training. Every accepted catalog item is eligible apart from previously seen items. Metrics are macro-averaged over 120 users with at least one novel test item. Precision uses K=5 as its denominator; recall uses each user's number of relevant items. NDCG uses binary relevance. Coverage is the fraction of catalog items recommended to at least one user; diversity is the mean fraction of recommendation pairs in different categories.

Model	Precision@5	Recall@5	NDCG@5	Coverage	Diversity
Popularity	0.1333	0.0684	0.1318	0.1200	0.8317
Personalized	0.2333	0.1229	0.2441	1.0000	0.6608

On this synthetic split, the personalized model has higher precision, recall, NDCG and catalog coverage, while the popularity model has higher category diversity. The generator deliberately embeds category preferences, so this result should not be generalized to actual users. Ratings are not predicted, so RMSE and MAE are not applicable.

Engineering trade-offs and next steps

Pandas and NumPy keep setup lightweight. CSVs make artifacts easy to inspect, while a custom file-based experiment tracker avoids a separate service. The trade-offs are shared latest outputs, no concurrency locking, and no scalable online feature store. Dense SVD and full-catalog scoring are suitable for this small example but not large production catalogs.

A single temporal split provides no uncertainty estimate; relevance treats all interactions as positive regardless of rating. Cold items, exposure bias, inventory, changing tastes and business constraints are not modeled. Before deployment, validate with real data and several time windows, assess cold-start segments and rating thresholds, introduce scalable sparse methods and monitoring, and compare online outcomes under a controlled experiment.

Reproducibility. Run `python dags/workflow.py` after installing requirements. Configuration, validation logs, feature catalog, comparison table and run-specific artifacts are included in the repository package.